

External Integrations

APIs, Databases & Webhooks for Agentic AI

Daz Akbarov

Solutions Architect

Amazon Web Services



Agenda

1. [Why Integrations Are the Whole Game](#)
2. [APIs — The Reach-Out](#)
3. [Databases — Memory & State](#)
4. [Webhooks — The Reach-In](#)
5. [Putting It Together](#)

Why Integrations Are the Whole Game

An Agent Without Integrations Is a Chatbot

LLM only

Generates text.
Lives in its training cut-off.

"I think you should..."

LLM + integrations

Reads live state.
Acts on real systems.

"I just did X. Result: Y."

The agent's capability is bounded by what it can **reach** and what it can **change**.

The Three Primitives

APIs

Reach out

Agent calls a service
and waits for an answer

Synchronous · request/response

Databases

Remember & query

Agent reads or writes
structured & vector state

Stateful · indexed

Webhooks

Get reached

External system pushes
events into the agent

Async · event-driven

Almost every "agentic capability" reduces to one of these three.

APIs — The Reach-Out

Tool Use Is the Foundation

What the LLM sees

A list of tools, each with:

- Name
- Description (the LLM reads this!)
- Parameters as JSON Schema
- Required vs optional fields

What the LLM produces

A structured tool call:

```
{  
  "tool": "get_weather",  
  "params": { "city": "Tokyo" }  
}
```

What the framework does

1. Match the call to a function
2. Validate parameters
3. Execute the function
4. Return result to the LLM
5. LLM continues reasoning

The LLM never executes code. It produces structured intent. Your harness does the work.

Defining a Tool — Strands Example

```
from strands import Agent, tool
import requests

@tool
def get_weather(city: str, units: str = "celsius") -> dict:
    """Get current weather for a city.

    Use when the user asks about temperature or conditions.
    """
    r = requests.get(
        "https://api.weather.example/v1/current",
        params={"q": city, "units": units},
        headers={"Authorization": f"Bearer {API_KEY}"},
        timeout=5,
    )
    return r.json()

agent = Agent(tools=[get_weather])
agent("Will I need an umbrella in Tokyo tomorrow?")
```

The docstring becomes the LLM's tool description. Type hints become the JSON Schema.

Authentication Patterns

API Keys

Bearer token in a header. Easiest. Use for low-risk reads.

```
headers={"Authorization": f"Bearer {KEY}"}
```

OAuth 2.0

User-delegated access. The agent acts *as* a specific user.

```
token = oauth_client.refresh(user.refresh_token)
```

IAM / SigV4

AWS-native. No long-lived secrets. Role-based.

```
boto3.client("dynamodb") # auth implicit
```

Signed Requests

HMAC over the request body. Used by webhooks too.

```
sig = hmac_sha256(secret, body)
```

Pick the **least-privilege** option that works. Never inject user secrets into prompts.

Reliability — Timeouts, Retries, Idempotency

```
from tenacity import retry, stop_after_attempt, wait_exponential

@tool
@retry(stop=stop_after_attempt(3),
      wait=wait_exponential(min=1, max=10),
      reraise=True)
def create_ticket(title: str, body: str, key: str) -> dict:
    """Create a support ticket. Safe to retry."""
    return requests.post(
        "https://api.ticketing.example/tickets",
        json={"title": title, "body": body},
        headers={"Authorization": f"Bearer {KEY}",
                "Idempotency-Key": key},
        timeout=10,
    ).json()
```

Timeouts prevent stuck loops. Always set them.

Retries handle transient failures. Exponential backoff.

Idempotency keys make retries safe. Required for writes.

MCP — Model Context Protocol

What it is

Open standard for connecting AI agents to external systems.

JSON-RPC over stdio, HTTP, or SSE.

Why it matters

- Anthropic, Nov 2024 → Linux Foundation, **Dec 2025**
- Adopted by ChatGPT, Cursor, Gemini, Copilot, VS Code
- **10,000+** public servers · **97M+** monthly SDK downloads

Anatomy

```
Agent -[stdio·HTTP·SSE] → MCP Server
                                ├── list_tools()
                                ├── call_tool()
                                └── resources & prompts
```

MCP — In Code

Connect a Strands agent to any MCP server in 8 lines.

```
from strands import Agent
from strands.tools.mcp import MCPClient
from mcp.client.sse import sse_client

mcp = MCPClient(lambda: sse_client("http://localhost:8000/sse"))
with mcp:
    agent = Agent(tools=mcp.list_tools_sync())
    agent("What issues are open on the platform repo?")
```

The agent doesn't know — or care — what's on the other end of the MCP connection. Postgres, GitHub, Slack, internal tool. Same protocol.

A2A — Agent-to-Agent

What it is

Open protocol for **agent-to-agent** communication. JSON-RPC over HTTPS. Each agent publishes an "agent card" describing what it can do.

Why it matters

- Google, April 2025 → Linux Foundation, **June 2025**
- 50+ partners — Atlassian, Salesforce, SAP, ServiceNow, MongoDB, Cohere
- Sits next to **MCP** in the Agentic AI Foundation

MCP vs A2A in one line

MCP — agent talks to *tools*.

A2A — agent talks to *other agents*.

```
# Discover what another agent can do
card = httpx.get(
    "https://billing.example/.well-known/agent.json"
).json()

# Delegate a task
result = a2a_client.tasks.send(
    agent_url="https://billing.example",
    task="Refund order 12345 — approved by support-agent"
)
```

When There's No API — Browser & Computer Use

The honest answer

Half the systems your agent needs to reach **don't have an API** — legacy intranets, vendor portals, admin UIs, government sites.

The toolkit (2026)

- **Anthropic Computer Use** — screenshots in, clicks/keys out
- **OpenAI Operator** — same idea, packaged
- **Playwright / Puppeteer** — code-driven browser
- **Browser Use, Skyvern, Stagehand** — agents on Playwright

```
# Computer-use shaped loop
while not done:
    screenshot = await browser.screenshot()
    action = await claude.decide(screenshot, goal)
    if action.type == "click":
        await browser.click(action.x, action.y)
    elif action.type == "type":
        await browser.type(action.text)
    elif action.type == "done":
        done = True
```

- ~3–5× the latency of an API call
- ~5–10× the token cost (vision frames)
- Brittle — UI changes break the agent

Use it when there's **no other way in**. Use APIs when there is.

Files — Read, Write, Persist

Where files show up

- **Coding agents** — read repo, edit, write tests
- **Research agents** — produce reports, slides, CSVs
- **Document agents** — extract from PDFs, fill forms
- **Pipeline agents** — staging artifacts in S3 between steps

Why files are their own primitive

Unlike API calls, files are **iterative and stateful**. The agent reads → edits → re-reads the same file 20 times in one task. The file *is* the working memory.

Sandbox the workspace — never let the agent write outside a chroot/jail.

Block path traversal —

`../../../etc/passwd` is the first thing to validate.

Diff, don't replace — for source code, prefer edit tools over full rewrites.

Files — Safe Tool Implementation

```
from pathlib import Path
WORKSPACE = Path("/sandbox/agent-workspace").resolve()

def safe(p: str) -> Path:
    full = (WORKSPACE / p).resolve()
    if not full.is_relative_to(WORKSPACE):
        raise ValueError("path traversal")
    return full

@tool
def read_file(path: str) -> str:
    """Read a text file from the workspace."""
    return safe(path).read_text()

@tool
def write_file(path: str, content: str) -> str:
    """Write a text file. Overwrites if exists."""
    p = safe(path)
    p.write_text(content)
    return f"wrote {len(content)} bytes to {p}"
```


Code Execution — When the Agent Builds Its Own Tool

Why this is different

Instead of calling a *fixed* tool, the agent **writes Python and runs it** in a sandbox. Useful when the question shape isn't predictable.

Where it shines

- Data analysis ("plot revenue by region")
- One-off transformations ("CSV to Parquet")
- Math, simulations, ad-hoc scripts
- Debugging — agent writes a probe to test a hypothesis

Never run agent-generated code in your main process. Always use an ephemeral, network-isolated, resource-capped sandbox.

Code Execution — Tool & Sandbox

```
@tool
def run_python(code: str) -> str:
    """Sandbox: 30s, 256MB, no network."""
    r = e2b.sandbox.run_code(
        code, language="python",
        timeout=30, memory_mb=256, network=False,
    )
    return (r.stdout + r.stderr)[:8000]
```

The sandbox *is* the security

- **E2B, Modal, Daytona** — managed sandboxes
- **AWS Lambda** with a strict role
- **Pyodide / WebAssembly** — in-browser, fully isolated

Databases — Memory & State

Two Database Roles for an Agent

Operational store

****Source of truth**** for the business

Customers · orders · inventory · users

SQL · DynamoDB · Mongo

Exact answers · transactions

Knowledge store

****Long-term memory**** for the agent

Docs · runbooks · past conversations

OpenSearch · pgvector · Bedrock KB

Semantic search · approximate match

Different access patterns, different risks, different tools.

Text-to-SQL — The Pattern

```
@tool
def query_orders(question: str) -> list[dict]:
    """Answer questions about the orders table."""
    schema = """
    CREATE TABLE orders (
        id UUID PRIMARY KEY, customer_id UUID NOT NULL,
        total_cents INTEGER NOT NULL,
        status TEXT CHECK (status IN ('pending', 'shipped', 'delivered')),
        created_at TIMESTAMPTZ NOT NULL
    );"""
    sql = llm_generate_sql(schema, question)    # second LLM call
    if not is_safe_select(sql):                 # SELECT only, allow-listed tables
        raise ValueError("Unsafe query rejected")
    return db.execute_readonly(sql).fetchall()
```

✓ Schema in the prompt — agent knows what columns exist

✗ Never expose `db.execute()` directly — always validate

Vector Search — Semantic Memory

How it works

```
@tool
def search_docs(question: str, k: int = 5) -> list[dict]:
    """Find documentation relevant to a question."""
    # 1. Turn the question into a vector
    embedding = bedrock.embed(
        model="amazon.titan-embed-text-v2",
        text=question,
    )

    # 2. Find vectors closest to it
    hits = opensearch.knn_search(
        index="docs",
        vector=embedding,
        k=k,
    )

    # 3. Return the source text + metadata
    return [{"text": h.text, "url": h.url} for h in hits]
```

When to use

- Questions over **unstructured** content
- Where the answer is similarity, not lookup
- Documentation, FAQs, past tickets, code

When **not** to use

- Exact-match lookups (use SQL)
- Strict counts or aggregations
- Anything where "wrong but close" is harmful

This is the foundation of **RAG**. Vector search → retrieved chunks → LLM answers grounded in them.

Knowledge Bases — The Managed Path

What you'd build by hand

1. Document ingestion pipeline
2. Chunking + embedding job
3. Vector store + index
4. Retrieval API
5. Re-ranking
6. Citation tracking

What you can buy instead

- **Bedrock Knowledge Bases** — point at S3 / Confluence / SharePoint / Salesforce
- **Vertex AI Search** — Google's equivalent
- **Azure AI Search** — Microsoft's
- **Pinecone Assistant, Weaviate** — vendor-managed RAG

Build vs buy: build for control & cost optimization, buy for speed.

Knowledge Bases — In Code

```
response = bedrock_agent_runtime.retrieve_and_generate(  
    input={"text": "What's our refund policy?"},  
    retrieveAndGenerateConfiguration={  
        "type": "KNOWLEDGE_BASE",  
        "knowledgeBaseConfiguration": {  
            "knowledgeBaseId": "ABCDEF1234",  
            "modelArn": (  
                "arn:aws:bedrock:us-east-1:"  
                "foundation-model/anthropic.claude-sonnet-4-v1:0"  
            ),  
        },  
    },  
)  
# Returns: generated answer + retrieved citations with S3 URIs
```

One API call replaces ingestion + chunking + embedding + retrieval + reranking + LLM grounding.

Webhooks — The Reach-In

Why Webhooks Make Agents Autonomous

Without webhooks

User has to **ask** the agent to do something every time.

The agent is reactive. Sleeps when no one types.

With webhooks

External systems **trigger** the agent.

GitHub PR opened	→ review agent runs
Stripe charge fails	→ recovery agent runs
PagerDuty alert	→ on-call agent runs
Slack @-mention	→ support agent runs

The agent is autonomous. Wakes up when reality changes.

Webhooks turn an agent from a tool you call into a teammate who watches.

Inbound Webhook Plumbing

```
# AWS Lambda receiving a GitHub webhook
import hmac, hashlib, os
SECRET = os.environ["GITHUB_WEBHOOK_SECRET"].encode()

def handler(event, context):
    body = event["body"]
    received = event["headers"]["x-hub-signature-256"]
    # 1. Verify the signature – REQUIRED
    expected = "sha256=" + hmac.new(SECRET, body.encode(), hashlib.sha256).hexdigest()
    if not hmac.compare_digest(received, expected):
        return {"statusCode": 401}

    # 2. ACK fast – providers retry on slow responses
    sqs.send_message(
        QueueUrl=AGENT_QUEUE, MessageBody=body,
        MessageDeduplicationId=event["headers"]["x-github-delivery"],
    )
    return {"statusCode": 200}
    # 3. Agent runs behind the queue – can take minutes
```

Outbound — When the Agent Sends Webhooks

Common patterns

- Post finding to **Slack** as a message
- Update a **Jira / ServiceNow** ticket
- Trigger a **GitHub Actions** workflow
- Notify a **PagerDuty** incident
- Push a metric to your **observability** stack

Treat them like API calls

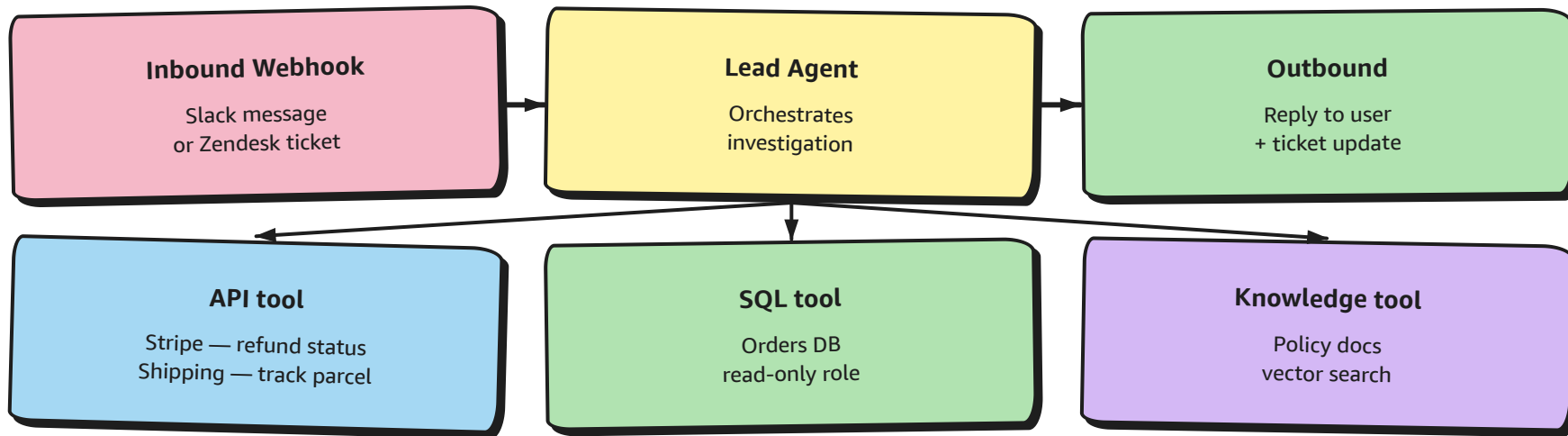
```
@tool
def post_to_slack(channel: str, message: str) -> dict:
    """Post a message to a Slack channel."""
    return requests.post(
        SLACK_WEBHOOK_URL,
        json={"channel": channel, "text": message},
        timeout=5,
    ).json()
```

Same retries, idempotency, signing rules apply.

Inbound webhooks **wake** the agent. Outbound webhooks **report** what it did.

Putting It Together

A Customer Support Agent



What Will Bite You — Security & Quality

Prompt injection via tool output

EchoLeak (CVE-2025-32711, CVSS 9.3)
— zero-click M365 Copilot exfil via crafted email.

GitHub MCP — Invariant Labs, May 2025 — malicious public issue exfiltrated private repos.

Treat tool output as untrusted.

Tool sprawl

Microsoft Research, 1,470 MCP servers
— accuracy can drop from 95% at 5 tools to **<30% at 100**.

Cliff, not slope. Cap each agent at ~10–20 tools.

Schema drift

DB column renamed; tool description still says the old name; agent silently picks the wrong column.

Generate schemas from the live source of truth.

OWASP LLM Top 10 — prompt injection ranked #1 in both 2023 and 2025.

What Will Bite You — Cost & Visibility

Cost explosion

One user request → 50 tool calls → 100 LLM calls → real money.

At frontier model prices, that's a couple of dollars *per request*.

At scale, you'll get a surprise bill.

Hard caps on iterations, total tokens, and per-tool budgets.

Circuit breakers when the agent loops.

Observability gaps

"It just stopped working" is the default failure mode.

Log every tool call — inputs, outputs, latency, success or error, trace ID.

OpenTelemetry is the standard. Strands, LangSmith, Helicone, Langfuse all wire in cleanly. Don't fly blind.

Your First Integration Project

Step 1 — One API

Build a weather assistant. One `@tool` . Real API key. Add a timeout. Test that it loops.

```
pip install strands-agents
```

Step 2 — Add a database

Spin up SQLite or Postgres. Add a `query_*` tool. Schema in the prompt. SELECT-only validator.

Step 3 — Add a webhook

Deploy a Lambda. Receive a Slack slash command. Verify signature. ACK in <1s. Queue to your agent.

Step 4 — Wrap as MCP

Turn one of your tools into an MCP server. Connect it from Claude Desktop. Now anyone can use it.

Each step is **one evening**. By the end of the week you've built a production-shaped agent.

Resources

SDKs & frameworks

- strandsagents.com — Strands Agents (Python + TS)
- [Bedrock AgentCore](#)
- [LangGraph](#)
- [OpenAI Agents SDK](#)

Standards

- [Model Context Protocol](#) — spec & SDKs
- [MCP Registry](#)
- [Agentic AI Foundation](#) — Linux Foundation home for MCP

Security & research

- [OWASP LLM Top 10](#) — prompt injection #1
- [EchoLeak case study \(arXiv 2509.10540\)](#)
- [Invariant Labs — GitHub MCP exploit](#)
- [Microsoft — Tool-space Interference](#)
- [Stripe — Designing APIs with idempotency](#)
- [AWS Builders' Library — Idempotent retries](#)

Reading

- [Building Effective Agents — Anthropic](#)
- [GitHub — Validating webhook deliveries](#)

Thank you!

The agent's capability is bounded by what it can reach.

[linkedin.com/in/daz-akbarov](https://www.linkedin.com/in/daz-akbarov)